

Inigo Quilez :: articles :: 3D models generation - 2005



When making a 4 kilobyte or a 64 kilobyte demo one needs to be imaginative in the way geometry is stored. In my experience, in a 4 kilobyte demo as [Kinderplumber](#) or [Stiletto](#) one cannot afford more than one kilobyte of geometry. In 64 kilobytes demos I guess something like twelve kilobytes is ok (as in [195/95/256](#) or [Paradise](#)). Of course, you can also use full procedural geometry and spend just few hundred bytes to build your scenes. You can also combine pure geometry with procedural techniques, and have the best of both worlds. So, let's quickly have an overview of some of the techniques I used with my colleagues at [rgba](#) for some of our demos.

Primitives

The simplest way to display some geometry is to use simple primitives. Like cubes, spheres, cylinders, toruses, planes. Many famous and successfull 4 kilobyte demos are done this way (*Atrium* and *Micropolis* being probably the best examples of this practice. Most 3d APIs like OpenGL or DirectX will give you some interface to render such primitives for free, but in any case, generating them is trivial and small.

Personally I have never being in favour of these methodes. It's boring, looks simple and has zero challenge. Also, in our productions, even if it seems we have being using them like the Image 1 taken from *195/95/256* might sugest, we have never done so. That scene shows a cube, a sphere, a plane, a cylinder and a cone, yet there is no code in the demo to generate such primitives. There are other techniques that can be used to generate such shapes without writing one single

line of extra code or using any external DLL or API. For example, revolution shapes.

Revolution shapes

Since the very beginning of my 64 kilobyte demo making experience, like with *Weektro* and *rare* back in 1998, I have always used revolution shapes to generate geometry.

Revolving geometry is a very cheap way to generate interesting geometry: given a 2d contour made of line segments, a surface is constructed as the 2d profile turns around a given axis. The only thing you need to store is few 2d coordinates for your profile (probably stored as 8 bit numbers) and then a little piece of code to generate triangles based on that data and few sinus and cosinus (around 300 bytes of code). The technique is quite flexible, it allows to build many types of objects, little columns, cups, mushrooms, tentacles, skydomes... Also, of course, cylinders, cubes and spheres.... So, that's why I never write one single byte of code to generate those primitives - if you have revolution shapes in your geometry engine, you have those ones too.

Clone, scale, rotate, bend

No need to say that you can get variations of your geometry by instantiating it many times (cloning) and changing the scale, orientation and bending of the clones. For example you can use procedural techniques to do so and build trees or cities, or you can even use more sophisticated algorithms to build something like *Atrium* or even manual placement of clones to shape a robot.

Displacement

However the technique I like the most is displacement of geometry. This very easily hides the simplicity of your data, and makes it look rich. We used it a lot both in *Paradise* and *195/95/256*. The idea is, once you have a 3d object, say a revolution shape, then process it's vertices one by one and apply some formula to it's vertex coordinates and move them. That way you can achieve scale, rotation and bending for free without any extra code, but also much more complex effects like fractal detail growth, ripples, bumps, etc. For example, in *Paradise* there are some mountains in the background of one of the scene. Those where not done with some special code for fractal mountains, but it was just a displacement modifier applied to a 2d horizontal plane. Another example are the columns shown on Image 3, taken from *195/95/256*, where the details of the columns where doing by displacement. Another example is visible in Image 1 - the potatoe in the background is a revolution object (a sphere) where the vertices where displaced with some perlin noise. Image 4 shows another use of displacement, in this case the teeths of the perimeter of the wheel where grown by displacement of vertices too.

Subdivision, softeing

Of course there are algorithms to tessellate geometry and smooth it. For example, we have used very often the well knwon Catmul Clark subdivision shceme to make surfaces more detailed and smooth. Catmul Clark works best with quadrilaterals; for pure triangular meshes you can try the Loop subdivision shema, although I really recommend Catmul Clark since most artists are used to it (that's what most modeling applications use today).

Also you can smooth a surface without generating more polygons. You can do that by low pass filtering the mesh, or in other words, doing some sort of blurring on the vertices. You

can see it like this: for a given vertex, find the vertices that are connected to it (through an edge), and do some averaging on vertex coordinates, and assign it to the coordinates of the current vertex. Just like you blur images.

Mesh compression

Of course, you can also make your 3d models in any 3d application and then export them to your demo. We have used this a lot in the last 4 and 64 kilobyte demos. Basically you normally want to store a low resolution version of your mesh, with as less polygons as possible, and store it as a C array of data in your demo. Usually vertices are quantified to 8 bits for example, and geometry converted to quad strips for compression. Well, there are many techniques to compress, other articles and presentations in this site explain the techniques in detail. You can easily get down to an average of 2 bytes per triangle in your models. For example, each of the statues in the *195/95/256* demo, like the one shown in Image 7, was compressed in around 750 bytes.

Metaballs

Another way to build smooth surfaces, although it's very difficult to control, are metaballs. I still have not seen a single demo where metaball based objects look anything near "not horrible", but hopefully somebody will improve the technique one day or another. The advantage is that once you place the metaprimatives (balls, cylinders, planes, whatever) either by manual placement by an artist or by procedural techniques (like ifs, fractals, or any other), you get a smooth surface without cracks. You can marching cubes or marching tetrahedra for that, the only problem is you will probably need

some simplification algorithm to reduce the huge amount of polygons that you will generate.

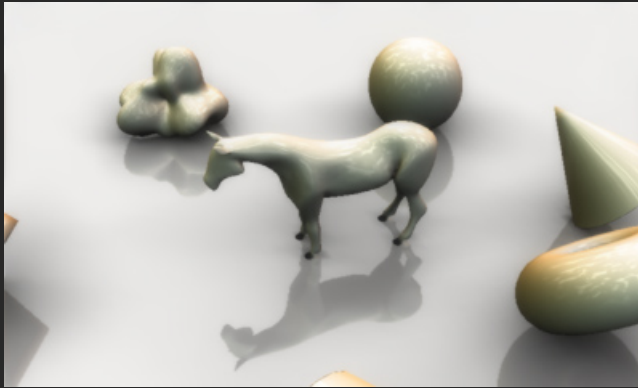


Image 1. A shoot from 195/95/256 showing some "primitives"...

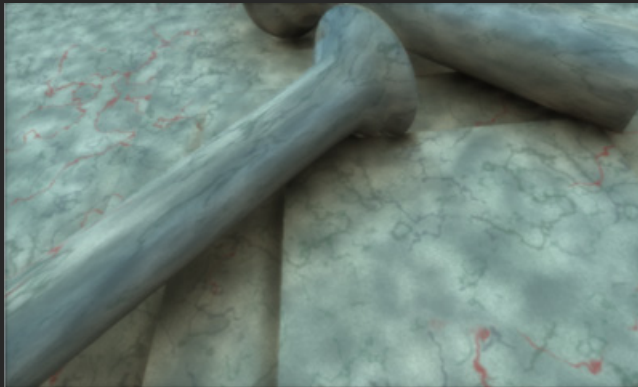


Image 2. A shoot from 195/95/256 showing revolution shapes



Image 3. Displacement applied to the geometry of Image 2



Image 4. Displacement used to grow some teeth in a wheel

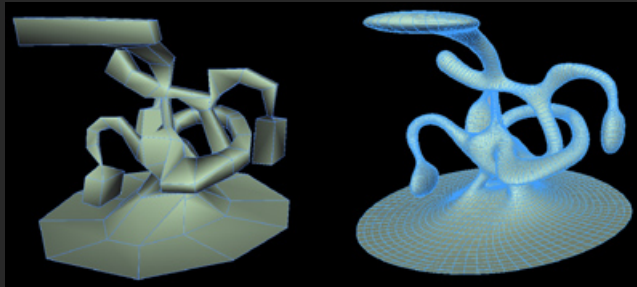


Image 5. A mesh before and after CC subdivision

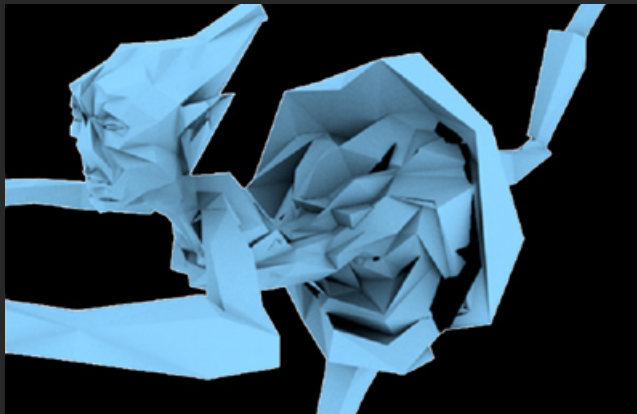


Image 6. A compressed mesh



Image 7. A compressed mesh after CC subdivision

inigo quilez - learning computer graphics since 1994